



武汉大学

WUHAN UNIVERSITY

数据结构与程序设计

(Python语言)

DATA STRUCTURE AND PROGRAM DESIGN IN PYTHON

第4讲 数据结构（一）



Python课程组



2025年02月



CONTENTS

目 录

- | | | | |
|---|------|---|--------------|
| 1 | 数据结构 | 5 | 列表推导式 |
| 2 | 列表 | 6 | 元组 |
| 3 | 列表切片 | 7 | 序列（含字符串常用方法） |
| 4 | 列表嵌套 | 8 | 实践指导 |



武汉大学
WUHAN UNIVERSITY

01

数据结构



引例



■ 学生成绩处理和统计

✱ 学习Python程序设计课程的某班20位学生参加了期中考试，任课教师希望能尽快了解本次考试的情况，那么用Python怎么处理考试成绩呢？

✱ 任课教师需要完成以下数据分析任务：

- (1) 存储所有学生成绩；
- (2) 计算班级平均分；
- (3) 找出最高分和最低分；
- (4) 统计以10分为一个区间的各分数段人数；
- (5) 快速查询某个学生的具体分数。



引例



■ 学生成绩处理和统计

✿ 如何有效的存储考试成绩?

➤ 简单罗列

使用变量存储（最原始的方式）

```
score1 = 91
```

```
score2 = 80
```

```
score3 = 76
```

```
...
```

```
score20 = 77
```

➤ 问题:

- 数据分散，无法批量操作；
- 统计计算需要逐个访问变量；
- 无法建立成绩与学生的关联。



引例



武汉大学
WUHAN UNIVERSITY

■ 学生成绩处理和统计

✿ 如何有效的存储考试成绩?

➤ 用“容器”保存成绩

91	80	76	78	64	88	75	69	...	77	<code>scores</code>
----	----	----	----	----	----	----	----	-----	----	---------------------

`scores[0]` `scores[1]`

...

`scores[n-1]`

`scores[i]` $i=0, 1, \dots, n-1$

用一个名字、下标运算符 `[ ]` 和下标去引用这些变量。

➤ 优点：数据集中管理，方便批量操作。

合理的数据结构设计尤为重要



数据结构



武汉大学
WUHAN UNIVERSITY

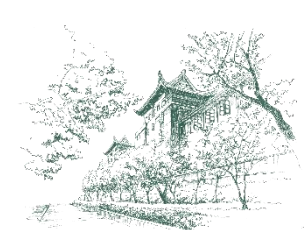
■ **数据结构** (Data Structure) 是计算机存储、组织数据的方式。

✿ 它是指相互之间存在一种或多种特定关系的数据元素的集合。

✿ 简单来说，数据结构定义了数据如何被存储、访问和操作，以便高效地解决各种问题。

■ 不同的数据结构适用于不同的场景，选择合适的数据结构可以大大提高程序的运行效率和代码的可读性。

■ 数据结构是算法设计的基础，不同的数据结构会影响算法的实现方式、运行效率与存储空间需求。

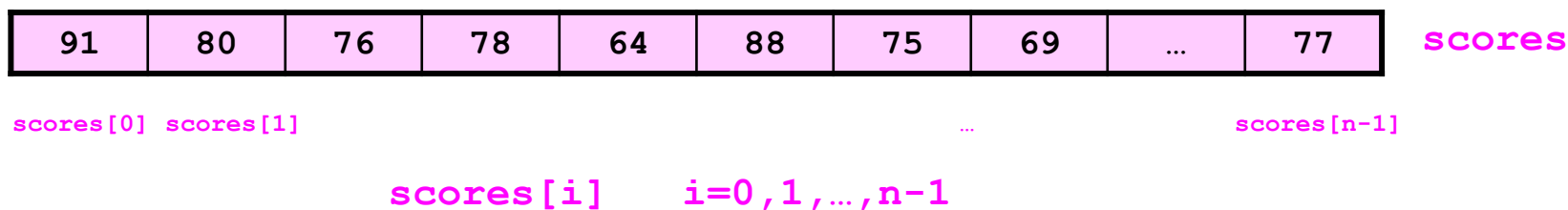


常用的数据结构



■ 数组

- ✱ 一种线性数据结构，它由一组相同类型的元素组成，在内存中占据连续的存储空间。
- ✱ 通过索引，能快速访问数组中的元素，时间复杂度为 $O(1)$ 。
 - 例如，在处理大量同类型的数值数据时，数组就非常适用。
- ✱ 但数组的大小通常在初始化时就已确定，后续若要增加或删除元素，可能需要重新分配内存，操作相对复杂。





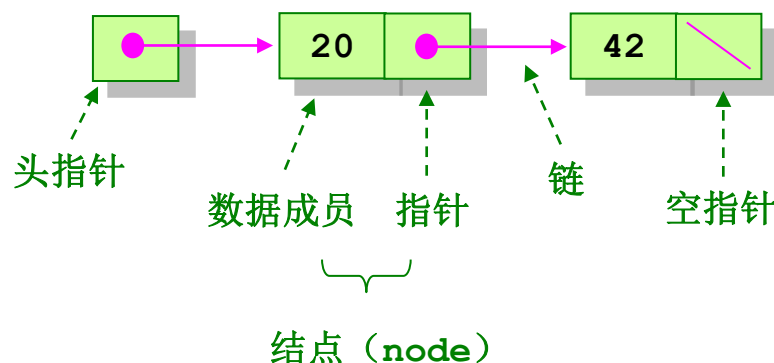
常用的数据结构



武汉大学
WUHAN UNIVERSITY

■ 链表

- ✱ 一种线性结构，与数组不同，链表的元素在内存中并不一定连续存储。
- ✱ 每个元素（节点）除了存储数据本身，还包含一个指向下一个节点的指针。
- ✱ 链表在插入和删除操作上比数组更具优势，时间复杂度为 $O(1)$ ，因为只需修改相关节点的指针即可。
- ✱ 不过，链表访问特定位置元素的效率较低，需要从头开始遍历，时间复杂度为 $O(n)$ 。





常用的数据结构



武汉大学
WUHAN UNIVERSITY

■ 栈

✱ 一种特殊的线性表，它遵循后进先出（LIFO, Last In First Out）的原则。

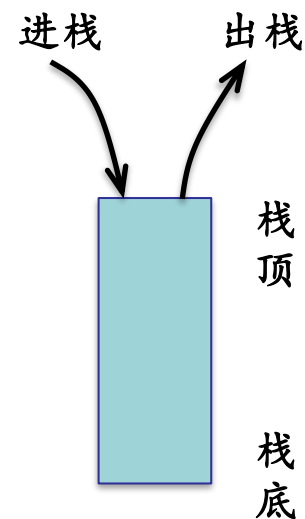
➤ 就像一摞盘子，最后放上去的盘子最先被拿走。

✱ 栈有两个主要操作：

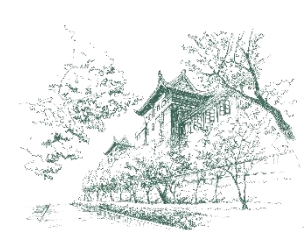
➤ 进栈（push），将元素添加到栈顶；

➤ 出栈（pop），移除栈顶元素。

✱ 在程序中，栈常用于实现递归算法、表达式求值等场景。



栈示意图



■ 队列

- ✱ 同样是线性表，与栈相反，它遵循先进先出（FIFO, First In First Out）的原则。
 - 类似于排队等待服务的场景。
 - 新元素在队尾加入（入队，**enqueue**），而元素从队头移除（出队，**dequeue**）。
- ✱ 队列在处理需要按顺序处理的任務，如任务调度系统中应用广泛。

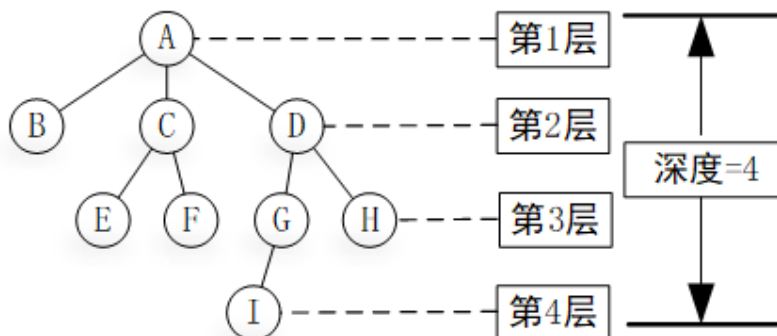


队列示意图



树

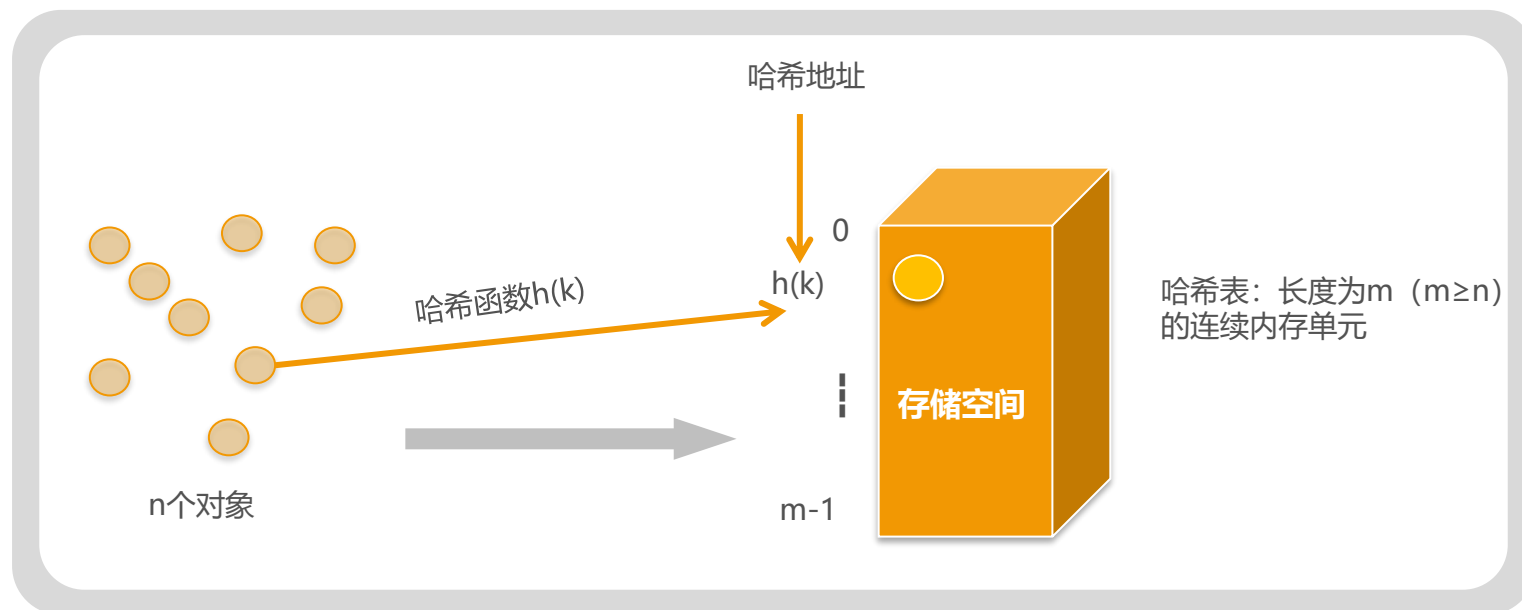
- ✱ 一种非线性数据结构，它由节点和边组成，具有层次结构。
- ✱ 树有一个根节点，每个节点可以有零个或多个子节点。
 - 二叉树是一种常见的树结构，每个节点最多有两个子节点。
- ✱ 树常用于组织和存储具有层次关系的数据，如文件系统的目录结构、数据库中的索引等。
- ✱ 通过树结构，我们可以高效地进行数据的查找、插入和删除操作，时间复杂度通常为 $O(\log n)$ ，比线性结构的某些操作更高效。





■ 哈希表也叫散列表

- ✱ 它通过哈希函数将键（用于唯一标识和查找数据的核心元素）映射到一个特定的存储位置，从而实现快速查找。
- ✱ 哈希表在理想情况下，查找、插入和删除操作的平均时间复杂度都能达到 $O(1)$ 。
- ✱ 哈希表常用于需要快速查找和存储数据的场景，如数据库索引、缓存系统等。





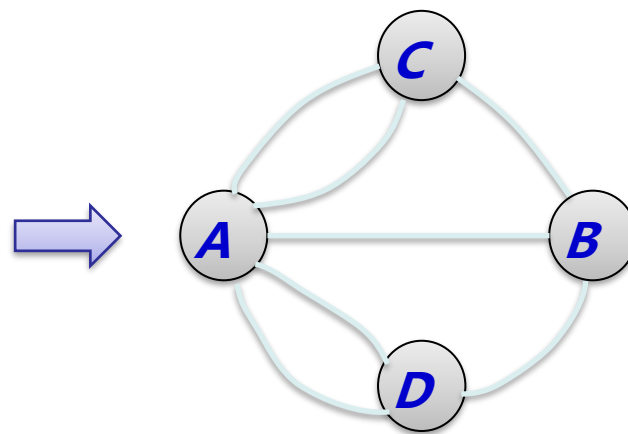
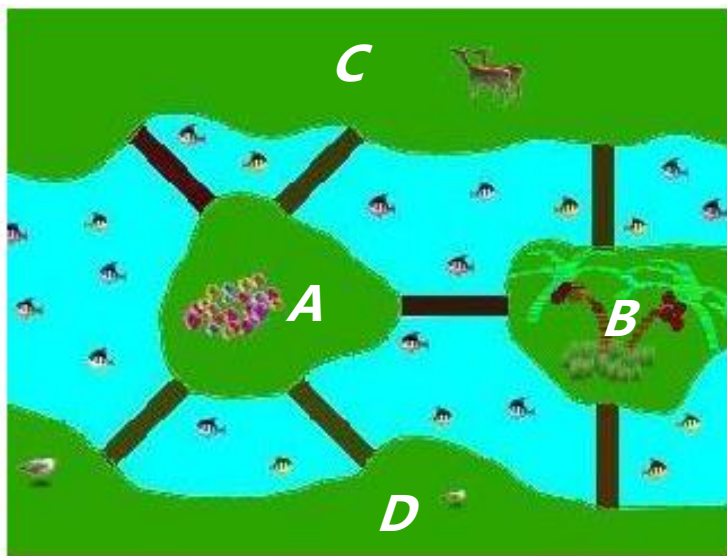
常用的数据结构



武汉大学
WUHAN UNIVERSITY

图

- ✱ 一种非线性的数据结构，用于表示对象之间的复杂关系。
- ✱ 它由一组顶点（Vertex，也称为节点）和连接这些顶点的边（Edge）构成。
- ✱ 边可以带有权重，用于表示顶点间关系的强度或代价等信息，并且边可以有向的（体现单向关系）或无向的（体现双向关系）。



图是现实问题的抽象



Python语言的数据结构



武汉大学
WUHAN UNIVERSITY

■ Python提供了丰富的数据结构来满足不同的编程需求。

■ 常用的数据结构

- ✱ 列表 (list) 是一种有序的可变序列。
- ✱ 元组 (tuple) 也是一种有序的序列，但元组是不可变的。
- ✱ 字符串 (str) 由字符组成的不可变序列。
- ✱ 字典 (dict) 是一种无序的键值对集合。
- ✱ 集合 (set) 是一个无序的、不包含重复元素的容器。



武汉大学
WUHAN UNIVERSITY

02

列表



列表

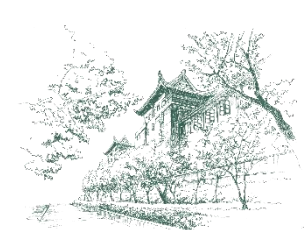


■ 列表

- ✿ 列表（类型名称为`list`）可以包含多个元素，
- ✿ 元素可以是任意类型的数据对象。

■ 列表的基本操作

- ✿ 创建列表
- ✿ 引用列表元素
- ✿ 增加元素
- ✿ 删除元素
- ✿ 修改元素
- ✿ 扩展列表
- ✿ 复制列表



创建列表



■ 通过直接写一个列表字面值来创建列表对象。

```
empty = [] # 创建一个空列表
fruits = ['苹果', '梨', '香蕉', '西瓜']
address = ['中国', '湖北省', '武汉市', '武昌区', '东湖南路8号']
student_info = ['赵元', '山西', '男', 2002, 1.75]

print(empty)
print(fruits)
print(address)
print(student_info)
```

```
[]
['苹果', '梨', '香蕉', '西瓜']
['中国', '湖北省', '武汉市', '武昌区', '东湖南路8号']
['赵元', '山西', '男', 2002, 1.75]
```

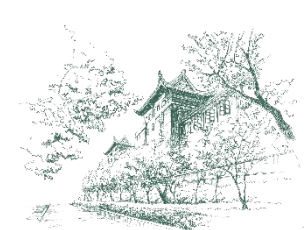


用内置函数操作列表



■ 操作列表的部分内置函数

- ✿ max、min、sum、len函数
- ✿ zip 函数用于将多个列表中元素重新组合为元组并返回包含这些元组的zip对象；
- ✿ enumerate 函数返回包含若干下标和值的迭代对象；
- ✿ map 函数把函数映射到列表上的每个元素；
- ✿ filter 函数根据指定函数的返回值对列表元素进行过滤；
- ✿ all 函数用来测试列表中是否所有元素都等价于True；
- ✿ any 用来测试列表中是否有等价于True的元素。



案例：学生成绩处理和统计



■ 基于列表的实现

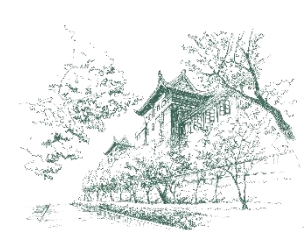
✿ 用列表保存学生成绩。

✿ 要计算前述的几个统计量，可以调用内置函数：

- **sum**函数用来计算列表中所有元素的和，
- **min**和**max**函数用来找出列表中的最小数和最大数。

```
scores = [91, 80, 76, 78, 64, 88, 75, 69, 87, 83,  
          90, 95, 86, 85, 87, 93, 92, 82, 88, 77]
```

```
count = len(scores)  
print(f"有{count}个学生的考试成绩: ", scores)  
average = sum(scores) / count  
print(f"平均成绩是{average}")  
min_score = min(scores)  
max_score = max(scores)  
print(f"最低分是{min_score}, 最高分是{max_score}")
```

创建列表

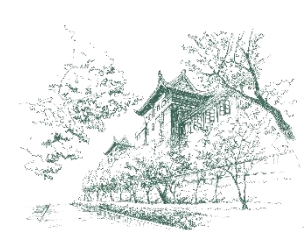


■ 也可以调用list函数来创建一个列表。

✱ 基于列表对象创建另一个内容相同的新列表

```
another = list(student_info)
print("学生信息: ", student_info)
print("复制信息创建另一个学生: ", another)
print(another is student_info)
```

```
学生信息:  ['赵元', '山西', '男', 2002, 1.75]
复制信息创建另一个学生:  ['赵元', '山西', '男', 2002, 1.75]
False
```

引用列表元素



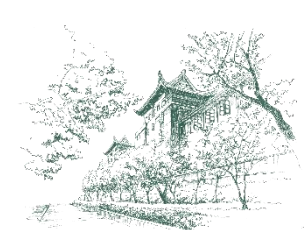
■ 用顺序号引用元素

- 列表中的元素是按照固定顺序排列的，可以根据要引用的元素的顺序号来引用元素。
- 列表中元素的顺序号从0开始，因此，scores列表元素的顺序号依次是0、1、2、.....、19。

```
scores = [91, 80, 76, 78, 64, 88, 75, 69, 87, 83,  
          90, 95, 86, 85, 87, 93, 92, 82, 88, 77]
```

```
print("第3个成绩是", scores[2])  
print(f"第6个成绩是 {scores[5]}")
```

```
第3个成绩是 76  
第6个成绩是 88
```



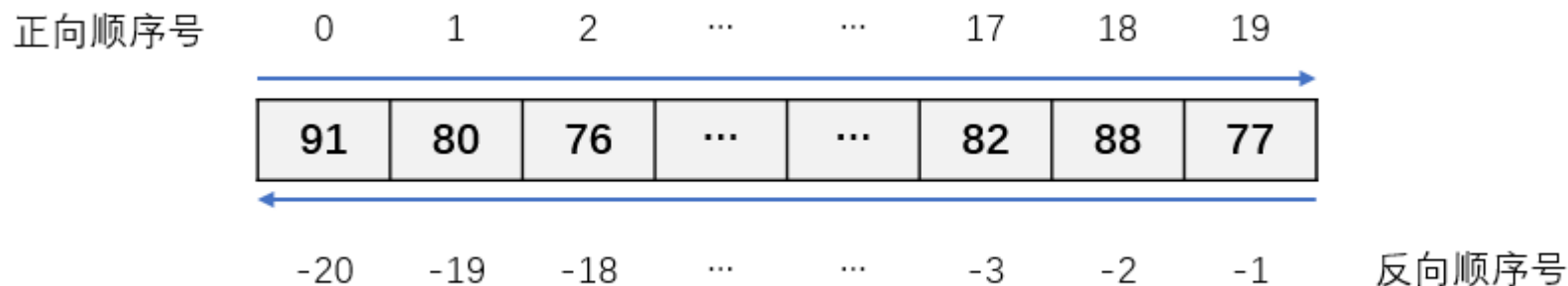
引用列表元素



武汉大学
WUHAN UNIVERSITY

■ 元素的索引

✿ 列表元素的顺序号可以从左往右数，也可以从-1开始从右往左数。



✿ 把引用列表元素的顺序号的几种称呼汇总一下，可以是元素在列表中的**顺序号**或**位置编号**、**下标**、**索引**。



引用列表元素



正确引用元素

✱ 在引用列表元素时，一定要确保元素的索引是有效的，否则会导致程序运行时错误。

```
print(scores[20])
```

```
-----  
IndexError                                Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_11008\3346254881.py in <module>  
----> 1 print(scores[20])  
  
IndexError: list index out of range
```



增加元素



■ 创建列表后，可以不断增加新元素。

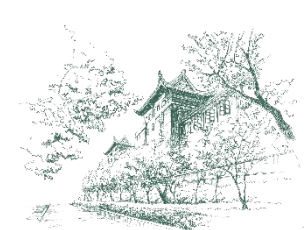
✿ 常用列表对象自己的方法 **append** 和 **insert** 增加新元素。

✿ **append** 方法是把新元素放到列表的末尾。

```
fruits = ['苹果', '梨', '香蕉', '西瓜']  
fruits.append('哈密瓜')  
print(fruits)  
fruits.append('葡萄')  
print(fruits)
```

✿ **insert** 方法是把新元素添加到指定位置。

```
student_info = ['赵元', '山西', '男', 2002, 1.75]  
student_info.insert(2, '太原')  
print(student_info)
```



案例：学生成绩处理和统计



■ 创建学生成绩表

✿ 依次输入学生成绩，并添加到列表。

```
''' 依次输入学生的成绩，并添加到成绩表 '''  
scores = [] # 创建列表对象，准备用来保存学生的成绩  
  
# 确定学生人数  
n = int(input("请输入学生人数: "))  
  
# 依次输入学生的成绩，并添加到成绩表  
for i in range(n):  
    score = int(input(f"请输入第{i + 1}个学生的成绩: "))  
    scores.append(score)  
  
# 显示成绩表  
print(scores)
```



删除元素



删除列表的元素

- 若要删除列表中的元素，可以使用`del`语句，

```
del student_info[2]  
print(student_info)
```

- 或者使用列表的`pop`和`remove`方法，
- 而列表的`clear`方法可以清空列表。

```
names.clear() # 使得names列表成为一个空列表。
```




删除元素



■ 用pop方法删除列表元素

✿ 如果想从列表中“取出”一个元素，可以使用pop方法。

✿ 该方法有两种使用形式：

➤ 删除列表末尾的元素，

```
fruit = fruits.pop()
print(fruits)
print(fruit)
```

➤ 删除任意位置的元素。

```
another = fruits.pop(1)
print(fruits)
print(another)
```



删除元素



■ 用remove方法删除列表元素

- ✱ 如果不知道要删除的元素的位置，但是知道它的值，那么就可以使用列表的remove方法。

```
fruits.remove('西瓜')  
print(fruits)
```

- ✱ 它只会删除第一个值匹配的元素。

```
names = ['赵元', '陈二', '田三', '赵元', '屈四', '王五']  
names.remove('赵元')  
  
print(names)    # 输出 ['陈二', '田三', '赵元', '屈四', '王五']
```



删除元素



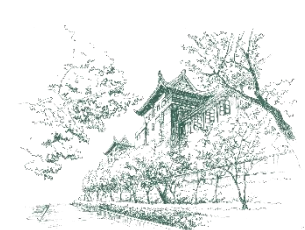
■ 用remove方法删除列表元素

✿ 如果没有找到匹配的元素，会出现ValueError错误，导致程序执行中断。

```
names.remove('秦六')
```

```
-----  
ValueError                                Traceback (most recent call last)  
~\AppData\Local\Temp\ipykernel_11008\712173424.py in <module>  
----> 1 names.remove('秦六')  
  
ValueError: list.remove(x): x not in list
```

✿ 思考：如何避免错误



修改元素



■ 修改列表元素引用的值

✱ 通过给列表元素赋值可以修改列表中的数据。

```
address = ['中国', '湖北省', '武汉市', '武昌区', '东湖南路8号']
```

```
address[-1] = '八一路299号'  
print(address)
```

```
['中国', '湖北省', '武汉市', '武昌区', '八一路299号']
```



扩展列表

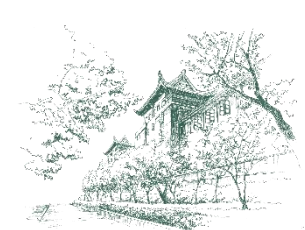


■ 扩展列表是指把另一个列表中的元素复制并添加到当前列表中。

✿ 通过调用列表对象的`extend`方法实现。

```
menu = ['鱼头泡饭', '清蒸大白刁', '排骨藕汤', '臭桂鱼']  
dishes = ['清炒洪山菜苔', '红糖糍粑']  
menu.extend(dishes)  
print(menu)
```

```
['鱼头泡饭', '清蒸大白刁', '排骨藕汤', '臭桂鱼', '清炒洪山菜苔', '红糖糍粑']
```



复制列表



复制列表对象

✿ 不能通过变量赋值实现列表的复制，而要使用列表的`copy`方法。

```
book = ['Python程序设计', '武汉大学出版社']  
new_book = book # 不能实现复制列表  
new_book[0] = 'C++程序设计'  
print(book)  
print(new_book)
```

```
['C++程序设计', '武汉大学出版社']  
['C++程序设计', '武汉大学出版社']
```

```
book = ['Python程序设计', '武汉大学出版社']  
new_book = book.copy() # 复制列表  
new_book[0] = 'C++程序设计'  
print(book)  
print(new_book)
```

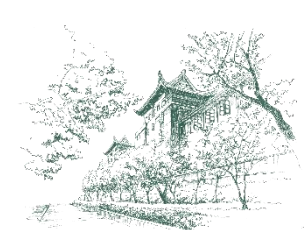
```
['Python程序设计', '武汉大学出版社']  
['C++程序设计', '武汉大学出版社']
```




列表对象的方法汇总表



方法	描述
L.append(object)	在列表L尾部追加对象。
L.clear()	移除列表L中的所有对象。
L.count(value)	计算value在列表L中出现的次数。
L.copy()	返回L的备份的新对象。
L.extend(Lb)	将Lb的表项扩充到L中。
L.index(value,[start, [stop]])	搜索value在列表L指定区间[start, stop)第一次出现的下标值。但在找不到时会引发 ValueError。
L.insert(index, object)	在列表L的下标为index的表项前插入对象。
L.pop([index])	返回并移除下标为index的表项，默认最后一个。
L.remove(value)	移除第一个值为value的表项。
L.reverse()	倒置列表L。
L.sort()	默认对列表中的数值按从低到高的顺序排序。

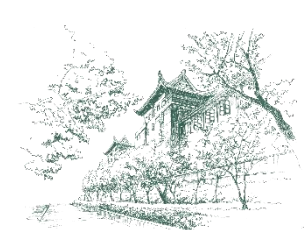


■ 列表排序和逆序操作

- ✿ 可以使用内置函数 **sorted**
- ✿ 也可以使用列表对象的排序方法 **sort**
- ✿ 可以使用列表的 **reverse** 方法对列表做逆序操作

```
sorted_scores = sorted(scores)
.....
sorted_scores.reverse()
.....
sorted_scores = sorted(scores, reverse=True)
.....
scores.sort(reverse=True)
```

```
words = ['python', 'program', 'language', 'design', 'computer', 'universal']
# 对列表words中的单词按字典顺序排序
words.sort()
print(words)
```



遍历列表



■ 遍历列表指的是按照某种顺序依次访问列表中的每一个元素。

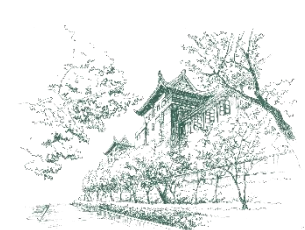
✿ 当需要对列表中的所有元素做某种处理时，就要遍历列表。

➤ 例如，要分段统计成绩，需要遍历保存成绩的列表。

✿ 常用while语句和for语句遍历列表。

```
cities = ['北京', '西安', '广州']  
  
print("我去过以下几座城市：")  
i = 0  
while i < len(cities):  
    print(cities[i])  
    i += 1
```

```
cities = ['北京', '西安', '广州']  
  
print("我去过以下几座城市：")  
for city in cities:  
    print(city)
```



for语句



■ for语句的一般写法

for 变量 in 可迭代对象:

循环体

[else:

子句]

✿ 用来遍历可迭代对象中的元素。

➤ 从一个可迭代的容器型数据对象中依次读取其元素，并赋值给指定变量。

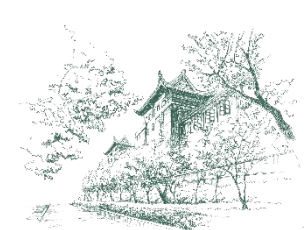
✿ 可迭代对象

- 序列：列表、字符串、元组等
- 字典、集合
- range、enumerate、zip、map等对象
- 文件
- 生成器

```
cities = ['北京', '西安', '广州']
```

```
print("我去过以下几座城市：")
```

```
for city in cities:  
    print(city)
```



案例：学生成绩处理和统计



完成考试成绩分段统计任务

参考实现一

```
scores = [91, 80, 76, 78, 64,
          88, 75, 69, 87, 83,
          90, 95, 86, 85, 87,
          93, 92, 82, 88, 77]

# .....

# 开始完成分段统计任务

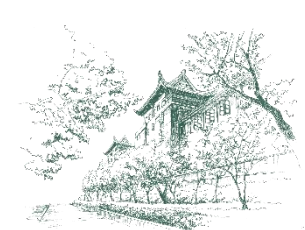
# 定义统计各分数段人数的变量，并设置初值0
count_1 = 0 # >=90
count_2 = 0 # 80~89
count_3 = 0 # 70~79
count_4 = 0 # 60~69
count_5 = 0 # <=59
```

```
# 遍历成绩列表，完成分段统计
for score in scores:
    if score >= 90:
        count_1 += 1
    elif score >= 80:
        count_2 += 1
    elif score >= 70:
        count_3 += 1
    elif score >= 60:
        count_4 += 1
    else:
        count_5 += 1

# 输出统计结果
print("各分数段人数如下")
print(">=90:", count_1)
print("80~89:", count_2)
print("70~79:", count_3)
print("60~69:", count_4)
print("<=59:", count_5)
```

各分数段人数如下

>=90:	5
80~89:	9
70~79:	4
60~69:	2
<=59:	0



■ 用内置函数range创建range对象（范围对象）

✿ 该函数的一般用法：

range(start, stop, step)

- 有range(stop)、range(start, stop)和range(start, stop, step)三种用法，参数start、stop和step必须是整数，start默认为0，step默认为1。
- 返回范围对象，包含左闭右开区间[start, stop)内以step为步长的一组整数。

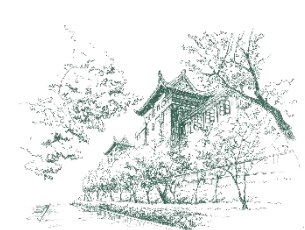
✿ 范围对象是一个可迭代对象。

```
for number in range(1, 6):  
    print(number)
```

```
1  
2  
3  
4  
5
```

```
# 用list函数把范围对象  
# 转换为列表对象  
print(list(range(10)))  
print(list(range(1, 10, 2)))  
print(list(range(9, 0, -1)))  
print(list(range(9, 0, -2)))  
print(list(range(-5, 6)))  
print(list(range(5, -4)))
```

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]  
[1, 3, 5, 7, 9]  
[9, 8, 7, 6, 5, 4, 3, 2, 1]  
[9, 7, 5, 3, 1]  
[-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5]  
[]
```



for语句与范围对象应用示例

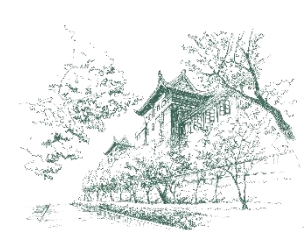


问题

✿ 使用嵌套的循环结构打印乘法口诀表。

```
"""打印乘法口诀表"""  
for i in range(1, 10):  
    for j in range(1, i+1):  
        print(f'{i}*{j}={i*j:<2}', end=' ')  
    print()
```

```
1*1=1  
2*1=2  2*2=4  
3*1=3  3*2=6  3*3=9  
4*1=4  4*2=8  4*3=12 4*4=16  
5*1=5  5*2=10 5*3=15 5*4=20 5*5=25  
6*1=6  6*2=12 6*3=18 6*4=24 6*5=30 6*6=36  
7*1=7  7*2=14 7*3=21 7*4=28 7*5=35 7*6=42 7*7=49  
8*1=8  8*2=16 8*3=24 8*4=32 8*5=40 8*6=48 8*7=56 8*8=64  
9*1=9  9*2=18 9*3=27 9*4=36 9*5=45 9*6=54 9*7=63 9*8=72 9*9=81
```

案例：学生成绩处理和统计

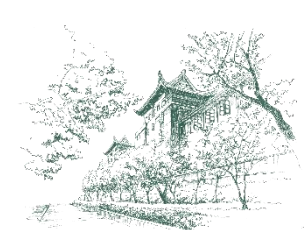


■ 完成考试成绩分段统计任务

✦ 改进：调整程序的数据结构

- 定义列表来保存一组统计量，同时用列表来保存各分段分割点的成绩。
- 灵活应对：分数段数量比较多，那么统计量的数量会增加，统计部分的分支数量也会增加。

```
scores = [91, 80, 76, 78, 64,  
          88, 75, 69, 87, 83,  
          90, 95, 86, 85, 87,  
          93, 92, 82, 88, 77]  
  
# .....  
  
# 用新方法完成分段统计任务  
  
# 定义列表保存各分数段人数，并设置初值0  
counts = [0, 0, 0, 0, 0]  
  
# 定义列表保存分数段分割点的成绩  
points = [90, 80, 70, 60]
```



案例：学生成绩处理和统计



完成考试成绩分段统计任务（改进版）

续

```
# 遍历成绩列表，完成分段统计
for score in scores:
    for i in range(len(points)): # 检查当前成绩在哪个分数段内
        if score >= points[i]: # 如果找到成绩落在的分数段，成绩 >= 分割点
            counts[i] += 1 # 统计量加一
            break # 结束检查
    else: # 如果成绩 < 最小的分割点，即最后一个分段
        counts[i+1] += 1 # 最后一个分段的统计量加一

# 输出统计结果
print("各分数段人数如下")
print(f">={points[0]}: {counts[0]}") # 输出第一个分段的名称和统计值，
                                     # 其名称与中间的段的命名规则不同
for i in range(1, len(points)): # 输出中间分段的名称和统计值
    print(f"{points[i]}~{points[i-1]-1}: {counts[i]}")
print(f"<={points[i]-1}: {counts[i+1]}") # 输出最后一个分段的名称和统计值，
                                           # 其名称与中间的段的命名规则不同
```

各分数段人数如下

>=90: 5

80~89: 9

70~79: 4

60~69: 2

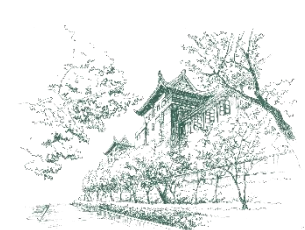
<=59: 0



武汉大学
WUHAN UNIVERSITY

03

列表切片



列表切片



武汉大学
WUHAN UNIVERSITY

■ 列表切片是列表元素的子集构成的列表。

seq[start : stop : step]

✱ seq是列表对象，从索引start开始，到索引stop之前，step是步长（默认为1）

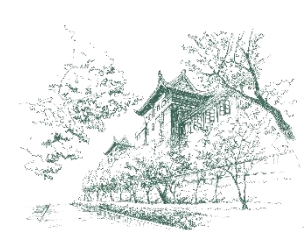
■ 创建切片

✱ 举例

```
cities = ['北京', '西安', '广州', '上海', '武汉', '深圳', '济南']  
slice1 = cities[1:4]  
print(slice1)
```



```
['西安', '广州', '上海']
```

列表切片

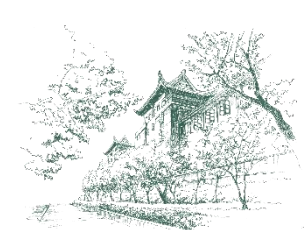


■ 创建切片

✿ 更多例子

```
cities = ['北京', '西安', '广州', '上海', '武汉', '深圳', '济南']
print("前3个城市: ", cities[:3])
print("后3个城市: ", cities[-3:])
print("所有城市: ", cities[:])
print(cities[:] is cities)
print("间隔着取: ", cities[::2])
print("倒着提取: ", cities[::-1])
```

```
前3个城市:  ['北京', '西安', '广州']
后3个城市:  ['武汉', '深圳', '济南']
所有城市:   ['北京', '西安', '广州', '上海', '武汉', '深圳', '济南']
False
间隔着取:   ['北京', '广州', '武汉', '济南']
倒着提取:   ['济南', '深圳', '武汉', '上海', '广州', '西安', '北京']
```

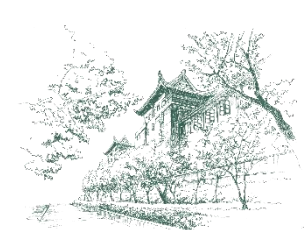


■ 用列表切片插入元素

✿ 以利用列表切片一次插入多个元素。

```
cities = ['北京', '西安', '广州', '上海', '武汉', '深圳', '济南']  
print(cities[3:3]) # 这也是一个切片，但是没有元素，  
                  # 作用是标识了列表中的一个位置，就是第4个元素的前面。  
cities[3:3] = ['长沙', '太原']  
print(cities)
```

```
[]  
['北京', '西安', '广州', '长沙', '太原', '上海', '武汉', '深圳', '济南']
```

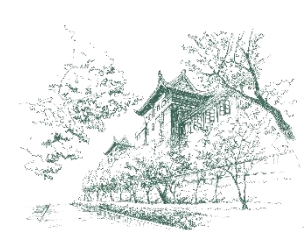


■ 用列表切片删除元素

✱ 通过给列表切片赋一个空列表，可以删除切片对应的元素。

```
letters = ['a', 1, 2, 3, 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k']  
letters[1:4] = []  
print("删除非字符串类型的元素后: ", letters)
```

```
删除非字符串类型的元素后:  ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h', 'i', 'j', 'k']
```

■ 用列表切片删除元素

✿ 用列表切片删除元素时，切片中的元素必须是位置连续的。

```
integers = [1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]
integers[1::2] = [] # 错误用法：试图删除非整数类型的元素，这些元素位置不连续
print(integers)
```

✿ 可以用pop方法配合while语句完成此任务。

```
integers = [1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]
i = 0
while i < len(integers):
    if isinstance(integers[i], float):
        integers.pop(i)
        continue
    i += 1
print(integers) # 输出: [1, 2, 3, 4, 5]
```



武汉大学
WUHAN UNIVERSITY

04

列表嵌套



列表嵌套



■ 创建元素是列表的列表

- ✿ 列表元素的类型可以是列表，即列表嵌套列表，也称为二维列表。
- ✿ 这种列表可以被用来按行保存二维数据表的数据，同时保留了二维数据表的数据结构。
- ✿ 例如
 - 如果有5位同学参加了3门计算机类课程的学习，并得到了3门课程的成绩，现在要编写程序对这些学生的成绩进行管理和统计分析。
 - 怎么设计保存这些成绩的数据结构呢？



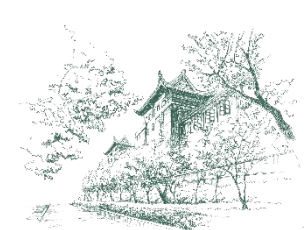
列表嵌套示例



■ 案例：用嵌套的列表来保存学生成绩

- ✱ 为了便于引用每个学生的成绩，可以用嵌套的列表来保存由这些成绩构成的成绩表。
- ✱ 每位学生的3门课程成绩保存在一个列表中，然后把5个结构相同的学生课程成绩列表作为元素保存在一个大列表中。

```
score_sheet = [  
    [85, 87, 80],  
    [93, 91, 88],  
    [88, 90, 87],  
    [82, 78, 80],  
    [91, 88, 85]  
]
```



嵌套列表元素的引用



引用嵌套列表的元素

例如

- `score_sheet[1]`是`score_sheet`列表的第2个元素，即第2位学生的成绩表，是一个列表：`[93, 91, 88]`。
- 如果想得到这位学生的第3门课的成绩，也就是`score_sheet[1]`的第3个元素，那就写`score_sheet[1][2]`。

```
score_sheet = [  
    [85, 87, 80],  
    [93, 91, 88],  
    [88, 90, 87],  
    [82, 78, 80],  
    [91, 88, 85]  
]
```




嵌套列表的应用示例



案例：按课程显示学生的成绩

显示每门课的学生成绩。

```
# 保存3门课的名称
course_names = ['Python', '数据分析', '机器学习']

# 保存5位学生3门课的成绩
score_sheet = [
    [85, 87, 80],
    [93, 91, 88],
    [88, 90, 87],
    [82, 78, 80],
    [91, 88, 85]
]

print("按课程显示学生的成绩\n")
for i in range(len(course_names)): # 按索引遍历课程，i是课程的索引
    print(f"{course_names[i]} 课程的成绩：") # 输出课程名称
    for j in range(len(score_sheet)): # 按索引遍历学生，j是学生的索引
        print(score_sheet[j][i]) # 输出索引为j的学生的索引为i的课程成绩
    print() # 在输出一门课程的所有成绩后输出一个空行
```

按课程显示学生的成绩

Python 课程的成绩：

85
93
88
82
91

数据分析 课程的成绩：

87
91
90
78
88

机器学习 课程的成绩：

80
88
87
80
85



嵌套列表的应用示例



思考：score_sheet的不足

- ✱ 如果要统计每位学生的总分，然后按总分进行排序，就需要在成绩表中增加学号和姓名。
- ✱ 怎样设计数据结构，以及如何按总分排序？

```
# 保存3门课的名称
course_names = ['Python', '数据分析', '机器学习']

# 保存5位学生3门课的成绩
score_sheet = [
    [85, 87, 80],
    [93, 91, 88],
    [88, 90, 87],
    [82, 78, 80],
    [91, 88, 85]
]
```

加入学号和
姓名

按总分从高
到低排序



嵌套列表的应用示例

案例：迷宫游戏（部分）

✱ 保存一个静态迷宫，然后显示迷宫。

```
#####  
@ #      #  
#      ####  
# #      #  
# # #    #  
# # #   ##  
#      #  
#####
```

迷宫

1	1	1	1	1	1	1	1
2	0	1	0	0	0	0	1
1	0	0	0	1	1	1	1
1	0	1	0	0	1	0	0
1	0	1	0	1	0	0	1
1	0	1	0	1	0	1	1
1	0	0	0	0	0	0	1
1	1	1	1	1	1	1	1

迷宫的数据



嵌套列表的应用示例



案例：迷宫游戏（部分）

代码

```
MAXROWS = 8 # 符号常量，迷宫行数
MAXCOLS = 8 # 符号常量，迷宫列数
```

```
# 用二维列表保存迷宫数据
```

```
maze = [
    [1,1,1,1,1,1,1,1],
    [2,0,1,0,0,0,0,1],
    [1,0,0,0,1,1,1,1],
    [1,0,1,0,0,1,0,0],
    [1,0,1,0,1,0,0,1],
    [1,0,1,0,1,0,1,1],
    [1,0,0,0,0,0,0,1],
    [1,1,1,1,1,1,1,1]
]
```

```
# 遍历列表元素（迷宫数据），并绘制（显示）迷宫
```

```
for row in maze:
    for cell in row:
        if cell == 0:
            print(' ', end='')
        elif cell == 1:
            print('#', end='')
        else:
            print('@', end='')
    print()
```



武汉大学
WUHAN UNIVERSITY

05

列表推导式



列表推导式



■ 列表推导式是一种基于一组数据生成一个新列表的方法。

✿ 编写一个列表推导式，执行该推导式就生成一个新列表。简单形式如下：

[表达式 for 变量 in 可迭代对象]

➤ 作用是遍历可迭代对象中的每个元素（赋值给变量），计算表达式的结果，生成新列表。

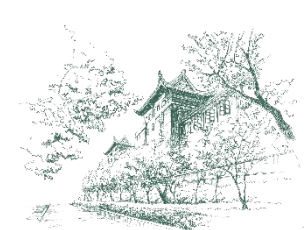
➤ 其中的表达式可以称为元素表达式。

✿ 例如：获取一个单词表中每个单词的长度，所有长度值保存在一个列表中。

```
words = ['python', 'program', 'language',  
         'design', 'computer', 'universal']  
lengths = [ ]  
for word in words:  
    lengths.append(len(word))  
print(lengths) # 输出: [6, 7, 8, 6, 8, 9]
```

```
words = ['python', 'program', 'language',  
         'design', 'computer', 'universal']  
  
lengths = [len(word) for word in words]  
  
print(lengths) # 输出: [6, 7, 8, 6, 8, 9]
```

简洁



列表推导式示例



■ 示例

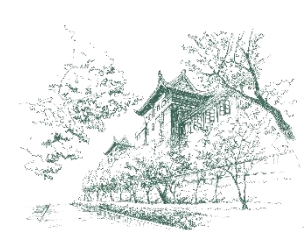
✿ 用列表推导式创建一个列表，包含所有一位正偶数的立方。

```
even_cubic = [even**3 for even in range(2, 10, 2)]  
  
print("所有一位正偶数的立方: ", even_cubic)
```

✿ 生成20个1~99的随机数。

```
import random  
  
random_numbers = [random.randint(1, 99) for i in range(20)]  
print("生成了20个随机数 (1~99): ")  
print(random_numbers)
```

➤ 列表推导式中**for**前面的元素表达式可以不引用**for**头部中的变量。



带筛选条件的列表推导式



■ 过滤不符合条件的元素

✿ 一般形式:

[元素表达式 for 变量 in 可迭代对象 if 条件表达式]

✿ 举例: 从生成的20个随机数中筛选出所有的奇数。

```
import random

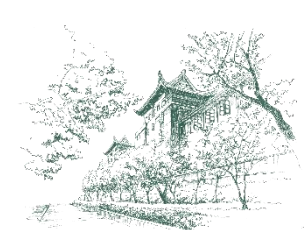
random_numbers = [random.randint(1, 99) for i in range(20)]
print("生成了20个随机数 (1~99): ")
print(random_numbers)
print("其中的奇数有: ")
print([number for number in random_numbers if number%2])
```

生成了20个随机数 (1~99):

[12, 41, 44, 20, 55, 86, 32, 62, 80, 49, 68, 82, 64, 47, 91, 27, 76, 81, 50, 93]

其中的奇数有:

[41, 55, 49, 47, 91, 27, 81, 93]



带筛选条件的列表推导式示例

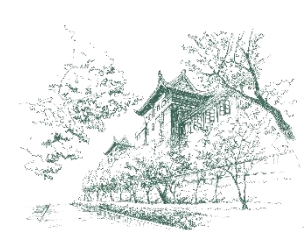


■ 过滤不符合条件的元素

✿ 举例：删除列表integers中非连续的浮点数。

```
integers = [1, 1.5, 2, 2.5, 3, 3.5, 4, 4.5, 5]
integers = [value for value in integers if isinstance(value, int)]
print(integers)  # 输出: [1, 2, 3, 4, 5]
```

➤ 要注意，这种方法创建了一个新列表对象。



有循环嵌套的列表推导式



遍历有嵌套结构的数据结构或生成组合

一般形式:

[元素表达式 for 变量1 in 可迭代对象1 for 变量2 in 可迭代对象2]

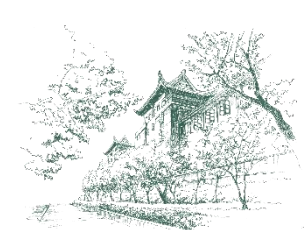
举例:

```
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
values = [v for row in matrix for v in row]
print(values) # 输出: [1, 2, 3, 4, 5, 6, 7, 8, 9]
```

```
students = ['陈琪', '李菲']
courses = ['Python', '数据分析']
pairs = [[student, course] for student in students for course in courses]
print(pairs) # 输出: [['陈琪', 'Python'], ['陈琪', '数据分析'], ['李菲', 'Python'], ['李菲', '数据分析']]
```

```
samples = [2, -3, 8, 7, -13, -5, 4]
methods = ['a', 'b']
array = [[m, s] for m in methods if m == 'b' for s in samples if s > 0]
print(array) # 输出: [['b', 2], ['b', 8], ['b', 7], ['b', 4]]
```

每一重for循环都可以附加筛选条件



生成多维列表



■ 元素表达式是一个列表推导式

✿ 举例：求矩阵的转置。

1	2	3	4
5	6	7	8
9	10	11	12

转置

1	5	9
2	6	10
3	7	11
4	8	12

```
matrix = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12] ]  
transposed_matrix = [[row[i] for row in matrix] for i in range(len(matrix[0]))]  
print(transposed_matrix) # 输出: [[1, 5, 9], [2, 6, 10], [3, 7, 11], [4, 8, 12]]
```

➤ 上面列表推导式的执行过程等价于下面的代码

```
matrix = [ [1, 2, 3, 4], [5, 6, 7, 8], [9, 10, 11, 12]]  
transposed_matrix = []  
for i in range(len(matrix[0])):  
    transposed_matrix.append([row[i] for row in matrix])  
print(transposed_matrix)
```



列表推导式应用说明



■ 列表推导式的元素表达式可以使用函数调用或复杂表达式

✿ 举例

```
def f(v):  
    if v%2 == 0:  
        v = v**2  
    else:  
        v = v+1  
    return v
```

```
print([f(v) for v in [2, 3, 4, -1] if v>0]) # 输出: [4, 4, 16]
```

```
print([v**2 if v%2 == 0 else v+1 for v in [2, 3, 4, -1] if v>0])  
# 输出: [4, 4, 16]
```



列表推导式应用说明



- 列表推导式通常比等效的for循环更快，因为底层用C语言优化。
- 对于简单逻辑，推荐使用列表推导式，但如果需要复杂逻辑，建议拆分为普通循环。

✿ 举例

```
print([(x, y) for x in [1, 2, 3] for y in [3, 1, 4] if x != y])  
# 输出: [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```

```
result = [ ]  
for x in [1, 2, 3]:  
    for y in [3, 1, 4]:  
        if x != y:  
            result.append((x,y))  
print(result)  
# 输出: [(1, 3), (1, 4), (2, 3), (2, 1), (2, 4), (3, 1), (3, 4)]
```



武汉大学
WUHAN UNIVERSITY

06

元组

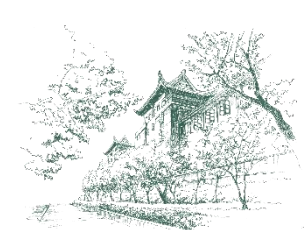


元组



■ 元组也是Python内置的组合数据类型之一，其类型名称为tuple。

- ✿ 在书写时，元组使用的是一对圆括号。
- ✿ 元组元素可以是任意类型，同一个元组可以包含不同类型元素。
- ✿ 元组与列表非常相似，也是包含多个元素的容器型数据对象，元素按顺序排列，并且位置固定，所以用引用列表元素的方法引用元组元素。
- ✿ 也可以对元组做切片操作。
- ✿ 但是，元组对象是不可变对象，即一旦创建一个元组，就不能增加、删除和修改元素。



元组字面值

举例

```
empty_tuple1 = () # 空元组
empty_tuple2 = tuple() # 用tuple函数创建空元组
one_item_tuple = ('only one',) # 只包含一个元素的元组，必须有一个逗号
rooms = ('101', '102', '103', '201', '202', '203')
sites = (('101', '102', '103'), ('201', '202', '203'))
python = ('Python', ('Guido van Rossum', '吉多·范罗苏姆', '龟叔'),
          ((1, 1991), (2, 2000), (3, 2008)))

print(empty_tuple1)
print(empty_tuple2)
print(one_item_tuple)
print(rooms)
print(sites)
print(python)
```




元组对象



武汉大学
WUHAN UNIVERSITY

■ 元组字面值

✱ 例如:

```
>>> t=(1,2,3)
```

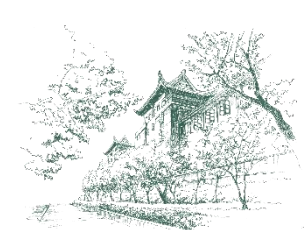
```
>>> t
```

```
(1, 2, 3)
```

```
>>> t=1,2,3
```

```
>>> t
```

```
(1, 2, 3)
```



元组字面值

例如，用字面表示方式创建元组

➤ 数据项可以是相同类型的，也可以是不同类型的。

```
>>> t2="east","south","west","north"
```

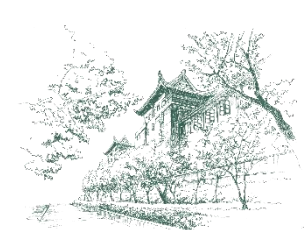
```
>>> t2
```

```
('east', 'south', 'west', 'north')
```

```
>>> t3="0010110","张山","men",18
```

```
>>> t3
```

```
('0010110', '张山', 'men', 18)
```



元组对象



元组字面值

例如：

```
>>> t5=23,(5,8,6),18,6
```

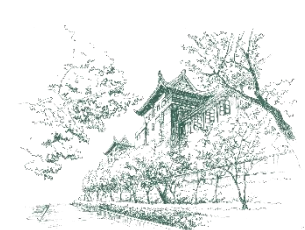
```
>>> t5
```

```
(23, (5, 8, 6), 18, 6)
```

```
>>> t6=t1,t2,t3,t4,t5
```

```
>>> t6
```

```
((1, 2, 3), ('east', 'south', 'west', 'north'), ('0010110', '张山', 'men', 18), (), (23, (5, 8, 6), 18, 6))
```



■ 元组的类型构造器

- ✿ 组合数据类型对象可以通过它们的类型构造器相互转化。
- ✿ 元组的类型构造器 `tuple()` 可以生成一个空的元组，也可以将字符串、列表、集合等转化为元组。

- ✿ 例如，生成一个空的元组

```
>>> t7=tuple()
```

```
>>> t7
```

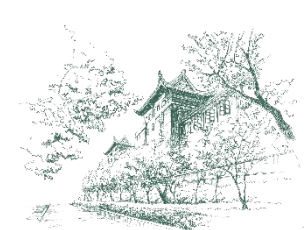
```
()
```

- ✿ 例如，将一个字符串转化为元组

```
>>> t7=tuple('Python')
```

```
>>> t7
```

```
('P', 'y', 't', 'h', 'o', 'n')
```



元组的基本操作



引用元组元素

- 可以通过下标引用元组中的某一项，称为元组元素。

- 下标从0开始。

- 举例：

```
>>> t6
```

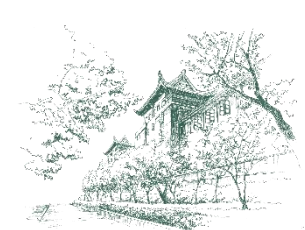
```
((1, 2, 3), ('east', 'south', 'west', 'north'), ('0010110', '张山', 'men', 18), (23, (5, 8, 6), 18, 6), ())
```

```
>>> t6[2]
```

```
('0010110', '张山', 'men', 18)
```

```
>>> t6[2][0]
```

```
'0010110'
```



元组的基本操作



武汉大学
WUHAN UNIVERSITY

引用元组元素

✱ 元组的元素不能被修改。

✱ 举例：

```
>>> t6[2]=t2
```

```
Trace back (most recent call last):
```

```
File "<pyshell#60>", line 1, in <module>
```

```
t6[2]=t2
```

```
TypeError: 'tuple' object does not support item assignment
```




武汉大学
WUHAN UNIVERSITY

07

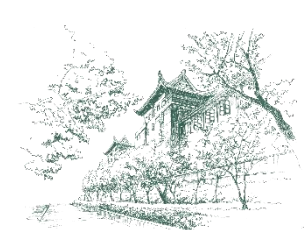
序列



序列



- 在Python中，序列是一种基础的数据结构，表示一组按特定顺序排列的元素集合。
- 序列的特点是元素有序、可通过索引访问，并支持通用操作（如切片、迭代等）。
- 内置的序列类型：列表、元组、字符串、字节串、字节数组等。
- 序列对象有一组公共的基本操作：引用元素、切片、复制元素、连接和解包赋值等，都有count和index方法。
- 因为序列对象是可选代的，所以许多内置函数都可以用来操作序列对象，例如sorted、len、max、min、map、enumerate、zip和filter等。



通过复制元素创建新序列



武汉大学
WUHAN UNIVERSITY

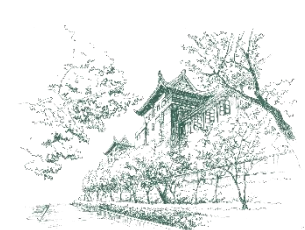
■ 一个序列和整数做乘法运算的表达式。

✿ 示例

```
values = (1, 2, 3)
new_values = values * 3
print(values)
print(new_values)
print(values is new_values)

message = "请重复3次" * 3
print(message)
```

```
(1, 2, 3)
(1, 2, 3, 1, 2, 3, 1, 2, 3)
False
请重复3次请重复3次请重复3次
```



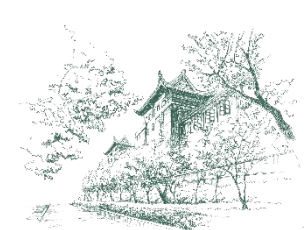
通过连接创建新序列

■ 两个同类型的序列对象，可以通过相加而合并两者的元素，并创建一个新序列。

✿ 示例

```
languages = ['Python', 'C', 'C++'] + ['C#', 'Java', 'SQL']  
print(languages)  
  
city = "Wuhan"  
print("Welcome to " + city + ".")
```

```
['Python', 'C', 'C++', 'C#', 'Java', 'SQL']  
Welcome to Wuhan.
```



序列解包赋值



■ 把一个序列对象赋值给多个变量，则首先会对序列进行解包，即把序列的元素取出来，然后依次赋值给多个变量。

✿ 示例

```
x, y, z = (1, 2, 3)
print(f"x={x}, y={y}, z={z}")

a, b, c = "abc"
print(f"a is {a}, b is {b}, c is {c}")
```

```
x=1, y=2, z=3
a is a, b is b, c is c
```

➤ 序列解包赋值时，序列中元素的数量应该和赋值运算符左边变量的数量一致。否则，就会导致错误。



序列解包赋值进阶用法



■ 使用 * 处理剩余元素

✿ 当需要解包的元素数量不确定时，可以用 * 捕获剩余元素为一个列表。

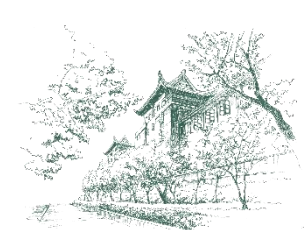
✿ 示例

```
first, *middle, last = [1, 2, 3, 4, 5]
print(first)      # 输出: 1
print(middle)     # 输出: [2, 3, 4]
print(last)       # 输出: 5

# 解包字符串
head, *tail = "Python"
print(head)       # 输出: P
print(tail)       # 输出: ['y', 't', 'h', 'o', 'n']
```

一个解包语句中只能有一个 *；* 变量后的变量必须显式指定。

```
x=1, y=2, z=3
a is a, b is b, c is c
```



序列解包赋值进阶用法



■ 嵌套解包

✿ 解包嵌套的序列结构。

✿ 示例

```
matrix = [  
    [1, 2, 3],  
    [4, 5, 6],  
    [7, 8, 9]  
]  
  
# 解包每一行  
(row1, row2, row3) = matrix  
print(row1)  # 输出: [1, 2, 3]  
  
# 进一步解包内部元素  
(a, b, c), (d, e, f), (g, h, i) = matrix  
print(a, d, g)  # 输出: 1 4 7
```




序列解包赋值进阶用法



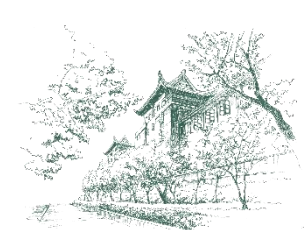
■ 忽略某些值

✱ 使用 `_` 占位符忽略不需要的值。

✱ 示例

```
# 忽略中间值
x, _, y = (10, "忽略我", 30)
print(x, y)  # 输出: 10 30

# 忽略多个值
a, *_ , b = [1, 2, 3, 4, 5]
print(a, b)  # 输出: 1 5
```



用count方法实现计数



count方法

- 用来统计一个数据在序列中出现的次数，或者说给定的数据与序列中多少个元素匹配。

`seq.count(v)`

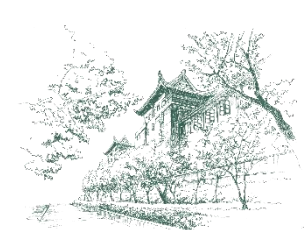
- 示例：用字符串的count方法统计某个字母、字符组合或词出现的次数。

```
introduction = "Python is a programming language that lets you work more quickly "\
               "and integrate your systems more effectively."

print("字母 a 出现的次数是", introduction.count('a'))
print("字符组合 ou 出现的次数是", introduction.count('ou'))
print("单词 more 出现的次数是", introduction.count('more')) # 结果符合预期
print("单词 our 出现的次数是", introduction.count('our'))   # 注意结果是不符合预期的
```

```
字母 a 出现的次数是 7
字符组合 ou 出现的次数是 2
单词 more 出现的次数是 2
单词 our 出现的次数是 1
```

要实现统计单词出现次数的任务，需要先对文本做分词处理，然后在词表中做查找和计数。



用count方法实现计数



count方法应用示例

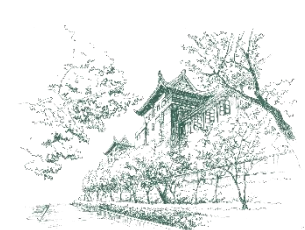
```
split_words = ['Python', 'is', 'a', 'programming', 'language',  
               'that', 'lets', 'you', 'work', 'more', 'quickly',  
               'and', 'integrate', 'your', 'systems', 'more', 'effectively.']  
print("单词 more 出现的次数是", split_words.count('more')) # 结果正确  
print("单词 our 出现的次数是", split_words.count('our'))   # 结果正确  
print()  
digits = (1,2,2,4,5,3,6,2,8,1,1,2,9,7,8,5,2,2,1,7,8,4,9,3,2,3)  
print("数字 9 出现的次数是", digits.count(9))  
print()  
values = [12, 3.4, 5, 67.8, 9, 0.12, 3.4, 5, 6.7, 89, 0.12, 34, 5.6]  
print("观测值 5 出现的次数是", values.count(5))  
print("观测值 3.4 出现的次数是", values.count(3.4))
```

对文本做了分词处理

单词 more 出现的次数是 2
单词 our 出现的次数是 0

数字 9 出现的次数是 2

观测值 5 出现的次数是 2
观测值 3.4 出现的次数是 2



用index方法查找匹配元素的索引

index方法

✿ 返回与指定数据匹配的元素的索引，如果没有匹配的元素就会报错。

`seq.index(v)`：在整个序列中，从第一个元素开始顺序比对，找到第一个匹配项立即结束

`seq.index(v, si)`：查找范围从索引为`si`的元素开始，直到最后一个元素

`seq.index(v, si, ei)`：查找范围从索引为`si`的元素开始，直到索引为`ei-1`的元素

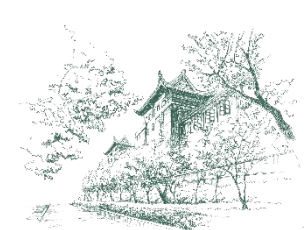
```
zen = "Beautiful is better than ugly."
print("字母 e 第一次出现的索引是", zen.index('e')) # 从第1个元素开始查找
print("从第3个字符开始，字符 e 出现的索引是", zen.index('e', 2)) # 从第3个元素开始查找
print("在第16个字符到第22个字符之前的范围内，字符 t 出现的索引是",
      zen.index('t', 15, 23)) # 在第16个元素到第22个元素之前的范围内查找，
print()
digits = (1,2,2,4,5,3,6,2,8,1,1,2,9,7,8,5,2,2,1,7,8,4,9,3,2,3)
print("数字 9 第一次出现的索引是", digits.index(9))
print()
values = [12, 3.4, 5, 67.8, 9, 0.12, 3.4, 5, 6.7, 89, 0.12, 34, 5.6]
print("观测值 5 第一次出现的索引是", values.index(5))
print("观测值 3.4 第一次出现的索引是", values.index(3.4))
print("观测值 56 第一次出现的索引是", values.index(56)) # ValueError: 56 is not in list
```

字母 e 第一次出现的索引是 1
从第3个字符开始，字符 e 出现的索引是 14
在第16个字符到第22个字符之前的范围内，字符 t 出现的索引是 15

数字 9 第一次出现的索引是 12

观测值 5 第一次出现的索引是 2
观测值 3.4 第一次出现的索引是 1

ValueError: 56 is not in list

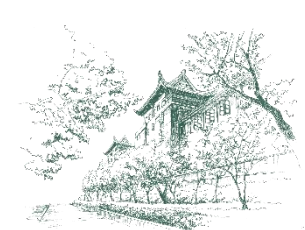


序列对象的类型转换



■ 类型转换

- ✿ 这里说的类型转换实际上是基于一种类型的序列对象创建另一种类型的序列对象。
- ✿ 例如，
 - 基于一个元组对象(12,34,56)调用内置函数list创建一个列表对象[12,34,56]。
- ✿ 因为两个对象的元素值相同，所以往往也称为对象的类型转换。
 - 元组对象和列表对象之间可以相互转换。
 - 用tuple和list函数可以把一个字符串对象转换为元组或列表。



序列对象的类型转换



类型转换

示例

```
tuple1 = (12, 34, 56)
list1 = list(tuple1) # 基于元组tuple1的元素新建一个列表
list1[2] = 78 # 修改列表的第3个元素值
print(tuple1)
print(list1)

tuple2 = tuple(['Python', '2022-2023-1', 3]) # 基于列表新建元组
print(tuple2)

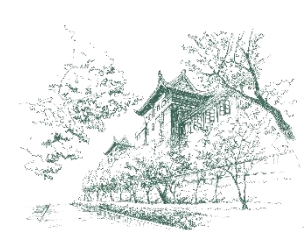
tuple3 = tuple("string") # 基于字符串的字符元素新建一个元组
print(tuple3)
```

```
(12, 34, 56)
[12, 34, 78]
('Python', '2022-2023-1', 3)
('s', 't', 'r', 'i', 'n', 'g')
```



■ 还有哪些对象属于序列

- ✿ Python的序列是一个具有广义性的概念。
- ✿ 用`range`函数创建的范围对象包含一组顺序排列的元素，而且可以遍历（可迭代的），因此也是序列。
- ✿ 那么，用`range`函数产生一组数据存储在范围对象中，然后调用`tuple`或`list`函数把范围对象转换为元组或列表，最后就可以用熟悉的方式去处理范围对象中的数据了。
- ✿ 用内置函数`map`、`enumerate`和`zip`等创建的对象都是序列，也可以被转换为元组或列表。

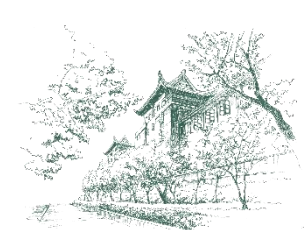


■ 还有哪些对象属于序列

✿ 举例

```
# 范围对象转换为列表
print(list(range(10, 21, 2)))
# map对象转换为列表
print(list(map(len, ('apple', 'banana', 'melon'))))
# 枚举对象转换为列表
print(list(enumerate(('apple', 'banana', 'melon'))))
# zip对象转换为列表
print(list(zip(('apple', 'banana', 'melon'), (5.99, 1.99, 2.99))))
```

```
[10, 12, 14, 16, 18, 20]
[5, 6, 5]
[(0, 'apple'), (1, 'banana'), (2, 'melon')]
[('apple', 5.99), ('banana', 1.99), ('melon', 2.99)]
```



字符串的常用方法



■ 字符串是不可变序列，提供丰富的内置方法处理文本数据。

- ✿ 拆分、连接
- ✿ 剪除
- ✿ 查找、替换
- ✿ 字母字符大小写转换
- ✿ 对齐
- ✿ 检测

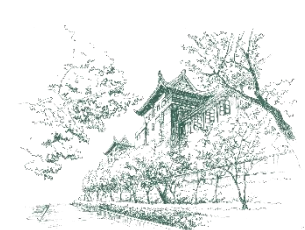


字符串的常用方法



字符串的常用方法（部分）

方法	功能描述	参数说明	返回值	代码示例
<code>split(sep, maxsplit)</code>	按分隔符从左向右拆分字符串	<code>sep</code> （分隔符，默认空格） <code>maxsplit</code> （最大分割次数，默认为-1）	列表	<code>"a,b,c".split(',') → ['a', 'b', 'c']</code>
<code>rsplit(sep, maxsplit)</code>	按分隔符从右向左拆分字符串	同上	列表	<code>"a,b,c".rsplit(',', 1) → ['a,b', 'c']</code>
<code>splitlines()</code>	按换行符拆分字符串	<code>keepends</code> （是否保留换行符，默认 False）	列表	<code>"line1\nline2".splitlines() → ['line1', 'line2']</code>
<code>join(iterable)</code>	将可迭代对象元素用字符串连接	<code>iterable</code> （需连接的元素集合）	新字符串	<code>"-".join(['2023', '08', '15']) → "2023-08-15"</code>
<code>strip(chars)</code>	去除字符串首尾的指定字符	<code>chars</code> （要删除的字符，默认空格）	新字符串	<code>" hello ".strip() → "hello"</code>
<code>lstrip(chars)</code>	去除字符串左侧的指定字符	同上	新字符串	<code>" hello".lstrip() → "hello"</code>
<code>rstrip(chars)</code>	去除字符串右侧的指定字符	同上	新字符串	<code>"hello ".rstrip() → "hello"</code>



字符串的常用方法



续

<code>replace(old, new, count)</code>	替换字符串中的子串	<code>old</code> (被替换的子串) <code>new</code> (新子串) <code>count</code> (替换次数, 可选)	新字符串	<code>"apple".replace('p', 'b', 1) → "abple"</code>
<code>find(sub, start, end)</code>	从左向右查找子串首次出现的索引(未找到返回-1)	<code>sub</code> (目标子串) <code>start/end</code> (搜索范围)	整数	<code>"python".find('th') → 2</code>
<code>rfind(sub, start, end)</code>	从右向左查找子串首次出现的索引(未找到返回-1)	同上	整数	<code>"apple".rfind('p') → 2</code>
<code>rindex(sub, start, end)</code>	与 <code>index()</code> 的不同之处是查找的方向	同上	返回整数或引发 <code>ValueError</code>	<code>"banana".rindex('a') → 5</code>
<code>upper()</code>	将字符串转为大写	无参数	新字符串	<code>"Hello".upper() → "HELLO"</code>
<code>lower()</code>	将字符串转为小写	无参数	新字符串	<code>"Python".lower() → "python"</code>

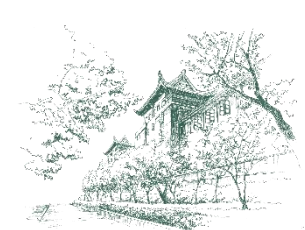


字符串的常用方法



续

<code>ljust(width, fillchar)</code>	左对齐字符串，右侧填充指定字符至目标宽度	<code>width</code> (目标宽度) <code>fillchar</code> (填充字符，默认空格)	新字符串	<code>"hi".ljust(5, '-') → "hi---"</code>
<code>rjust(width, fillchar)</code>	右对齐字符串，左侧填充指定字符至目标宽度	同上	新字符串	<code>"42".rjust(5, '0') → "00042"</code>
<code>center(width, fillchar)</code>	居中对齐字符串，两侧填充指定字符	同上	新字符串	<code>"text".center(10, '*') → "***text***"</code>
<code>zfill(width)</code>	左侧用零填充字符串至指定长度	<code>width</code> (目标长度)	新字符串	<code>"7".zfill(3) → "007"</code>
<code>startswith(prefix)</code>	检查字符串是否以指定前缀开头	<code>prefix</code> (前缀字符串或元组)	布尔值	<code>"data.csv".endswith('.csv') → True</code>
<code>endswith(suffix)</code>	检查字符串是否以指定后缀结尾	<code>suffix</code> (后缀字符串或元组)	布尔值	<code>"data.csv".endswith('.csv') → True</code>
<code>isdigit()</code>	判断字符串是否全为数字字符	无参数	布尔值	<code>"123".isdigit() → True</code>
<code>isalpha()</code>	判断字符串是否全为字母字符	无参数	布尔值	<code>"Hello".isalpha() → True</code>



字符串的常用方法



■ 方法特性总结

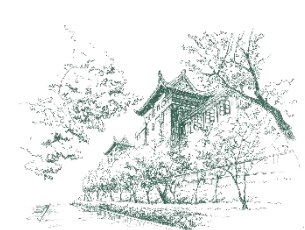
- ✿ **不可变性：**所有涉及改变文本内容的方法均返回新字符串，原字符串不变。
- ✿ **链式调用：**支持连续操作。
- ✿ **split()/strip()**等方法sep/chars参数支持多字符。
- ✿ **startswith()/endswith()**可传入元组，同时匹配多个前缀/后缀。

```
s = "Python"
s_new = s.upper() # 原字符串s仍为"Python"
print(s) # 输出: Python

print(" Hello, World! ".strip().lower()) # 输出: hello, world!

print("*-*Hello, World!*-*".strip('-*')) # 输出: Hello, World!

print("image.png".endswith(('.png', '.jpg')) # 输出: True
```



字符串的常用方法



■ 应用场景示例

```
# 数据清洗：去除多余空格并标准化格式
raw_data = "  姓名：吴昕；年龄：25；成绩：95  "
cleaned = raw_data.strip().replace(';', ',').lower()
print(cleaned)  # 输出：姓名：吴昕，年龄：25，成绩：95

# 文件扩展名验证
filename = "report.pdf"
if filename.endswith(('.pdf', '.docx')):
    print("支持的文件格式")
# 输出：支持的文件格式

# 提取URL中的域名
url = "https://www.whu.edu.cn/index"
domain = url.split('://')[1].split('/')[0]
print(domain)  # 输出：www.whu.edu.cn
```




武汉大学
WUHAN UNIVERSITY

08

实践指导



应用案例1：动态构建二维列表

■ 用嵌套列表来保存若干学生3门课程的考试成绩

✿ 在输入数据时构建保存在嵌套列表中二维成绩表的过程：

- 起初，`score_sheet`是一个空列表；
- 在每次输入一个学生的3门课程成绩前，新建一个空列表`student_scores`，准备用来保存3个成绩；
- 每输入一个成绩，马上添加到`student_scores`；
- 一个学生的3个成绩输入后，把当前的`student_scores`添加到成绩表，即`score_sheet`；
- 然后从第二步重复执行上述步骤，直到输入了所有学生的成绩。



应用案例1：动态构建二维列表



■ 在输入数据时构建保存在嵌套列表

```
# 保存3门课的名称
course_names = ['Python ', '数据分析', '机器学习']

# 保存5位学生3门课的成绩
score_sheet = []

# 输入成绩，每次输入一个学生的3个成绩
for i in range(5):
    print(f"输入第{i+1}位学生的考试成绩")
    student_scores = [] # 创建空列表，用来保存第i+1位学生的3个成绩
    for name in course_names:
        score = eval(input(f"{name}: "))
        student_scores.append(score)

    # 把保存一个学生3个成绩的列表作为元素添加到成绩表
    score_sheet.append(student_scores)

# 输出现在的成绩，检查是否正确保存了数据
print("\n成绩表: ", score_sheet)
```



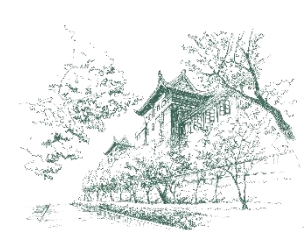
标准库的statistics模块



statistics模块

- 通过调用statistics模块中的函数，可以方便地计算给定数据集的平均值、中位数、众数、方差和标准差等统计量。
- 这些统计量可以帮助了解和描述数据集的特征。
- 示例

课程名称	平均值	中位数	众数	方差	标准差
Python	87.8	88.0	85.0	19.7	4.4
数据分析	86.8	88.0	87.0	26.7	5.2
机器学习	84.0	85.0	80.0	14.5	3.8



应用案例2：成绩统计



■ 用statistics模块计算各门课程的统计量

```
import statistics as stat

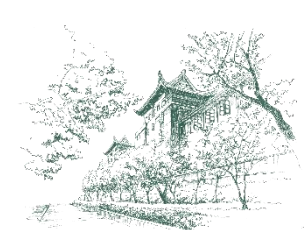
# 这里省略了输入成绩的代码
# .....

# 第一步：遍历成绩表构造一个按课程组织成绩的二维列表

# 先用列表推导式一个包含3个空列表元素的列表，3个空列表对应3门课程的成绩列表
course_score_sheet = [[] for course in course_names]

# 按学生遍历成绩表，把每个学生的3个成绩添加到对应课程的成绩列表中
for student_scores in score_sheet:
    for i in range(len(student_scores)):
        course_score_sheet[i].append(student_scores[i])

# 为了帮助阅读代码，在这里输出course_score_sheet，正式发布程序时，可以注释下面一行代码
# print(course_score_sheet)
```



应用案例2：成绩统计



■ 用statistics模块计算各门课程的统计量

✱ 续

第二步：计算各门课程的统计量

```
print("课程名称 平均值 中位数 众数 方差 标准差")
for i in range(len(course_names)):
    course_scores = course_score_sheet[i]
    mean = stat.mean(course_scores) # 平均值
    median = stat.median(course_scores) # 中位数
    mode = stat.mode(course_scores) # 众数
    variance = stat.variance(course_scores) # 方差
    stdev = stat.stdev(course_scores) # 标准差

    print(f"{course_names[i]} {mean:<6.1f} {median:<6.1f} {mode:<6.1f} {variance:<6.1f} {stdev:<6.1f}")
```



用NumPy进行数值计算

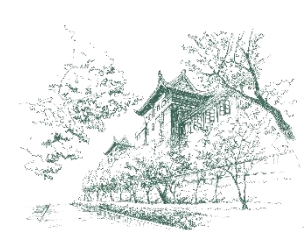
NumPy

✿ 是Python中用于科学计算的基础库。

✿ <https://numpy.org/>

✿ 它提供了

- 多维数组对象（如`ndarray`）、
- 多种派生对象（如掩码数组和矩阵），
- 以及用于快速操作数组的通用函数（包括数学运算、逻辑运算、形状操作、排序、选择、I/O、离散傅里叶变换、基础线性代数、基础统计操作和随机模拟等）。



用NumPy进行数值计算

■ NumPy的核心是ndarray对象，它是封装了同构数据类型的n维数组。

属 性	说 明
ndim	返回 int，表示秩（轴的数量或数组的维数）。一维数组的秩为 1，二维数组的秩为 2，以此类推
shape	返回 tuple，表示数组形状的元组，对于 n 行 m 列的二维数组，形状为 (n,m)
size	返回 int，数组元素的总个数，等于 shape 各维度值的乘积
dtype	返回数组元素的类型
itemsize	返回 int，数组中每个元素的大小（以字节为单位）



用NumPy进行数值计算



武汉大学
WUHAN UNIVERSITY

■ NumPy的核心是ndarray对象，它是封装了同构数据类型的n维数组。

```
>>> import numpy as np
```

导入NumPy（需确保已经安装NumPy库）

```
>>> a = np.arange(15).reshape(3, 5)
```

使用np.arange()函数创建3行5列的数组

```
>>> a
```

```
array([[ 0,  1,  2,  3,  4],
       [ 5,  6,  7,  8,  9],
       [10, 11, 12, 13, 14]])
```

```
>>> a.ndim
```

显示数组的维数

```
2
```

```
>>> a.shape
```

显示数组的形状

```
(3, 5)
```

```
>>> a.size
```

显示数组元素的个数

```
15
```

```
>>> a.dtype.name
```

显示数组元素的类型名

```
'int64'
```

```
>>> a.itemsize
```

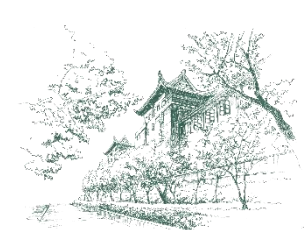
显示数组元素的字节数

```
8
```

```
>>> type(a)
```

显示a的类型

```
<class 'numpy.ndarray'>
```



用NumPy进行数值计算



武汉大学
WUHAN UNIVERSITY

■ NumPy的核心是ndarray对象，它是封装了同构数据类型的n维数组。

```
>>> b = np.array([1.2, 3.4, 5.6])
```

使用np.array()函数将Python列表转换为数组

```
>>> b
```

```
array([1.2, 3.4, 5.6])
```

```
>>> b.ndim
```

显示数组的维数

```
1
```

```
>>> b.dtype
```

显示数组元素的类型

```
dtype('float64')
```

- 除了np.arange()和np.array()函数外，Numpy还提供了多种创建数组的方式，如创建一维等差数列数组、一维等比数列数组、未初始化的数组、全0数组、全1数组、特定值填充的数组、对角线数组、单位矩阵以及各种分布的随机数组等。
- 与Python序列类似，数组可被索引、切片和迭代。
- 此外，可对数组进行形状操作（包括组合与分割），以及使用通用函数进行更多操作。



应用案例3：用NumPy处理图像



图像处理

- ✱ 图像本质上是一个多维数组（通常是二维或三维数组），
- ✱ 通过NumPy，可以轻松地对图像进行裁剪、翻转、调整亮度及滤镜应用等。

图像加载

```
import numpy as np
from PIL import Image                                # 导入Image模块（需确保已经安装了Pillow库）

# 读取图像
image = Image.open('whu_lib.jpg')
# 显示图像
image.show()

# 将图像对象转换为NumPy数组
im_array = np.array(image)

# 输出数组的形状和数组元素类型（图像维度为高度、宽度和RGB通道，取值范围为：0-255）
print(im_array.shape, im_array.dtype)
# 输出数组im_array,
print(im_array)
```



应用案例3：用NumPy处理图像



武汉大学
WUHAN UNIVERSITY

■ 图像加载

```
(637, 943, 3) uint8
[[[144 204 254]
  [144 204 254]
  [146 204 254]
  ...
  [133 198 252]
  [133 198 254]
  [135 197 254]]

[[[143 203 253]
  [143 203 253]
  [145 203 253]
  ...
  [134 199 253]
  [136 198 255]
  [136 198 255]]

...
...
[ 55  52  37]
[ 64  60  48]
[ 57  53  42]]]
```





应用案例3：用NumPy处理图像



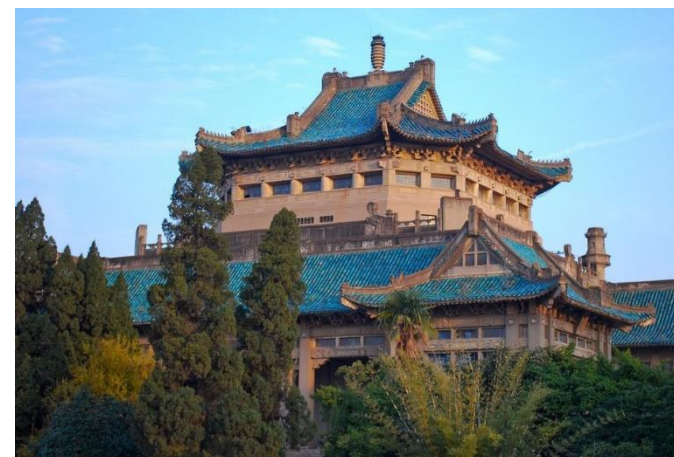
武汉大学
WUHAN UNIVERSITY

图像裁剪和翻转

```
# 定义裁剪区域（行范围：row_start:row_end，列范围：column_start: column_end）
row_start, column_start = 20, 120
row_end, column_end = 300, 730
# 裁剪
cropped_im = im_array[row_start:row_end, column_start:column_end]
Image.fromarray(cropped_im).show()

# 水平翻转，反转第2个维度
flipped_im_x = im_array[:, ::-1]
Image.fromarray(flipped_im_x).show()

# 垂直翻转：反转第1个维度
flipped_im_y = im_array[::-1]
Image.fromarray(flipped_im_y).show() # 显示处理后的图像
Image.fromarray(flipped_im_y).save("output.png") # 保存处理后的图像
```





应用案例3：用NumPy处理图像



武汉大学
WUHAN UNIVERSITY

灰度滤镜

```
gray_im = np.dot(im_array, [0.2989, 0.5870, 0.1140]).astype(np.uint8)  
Image.fromarray(gray_im).show()
```





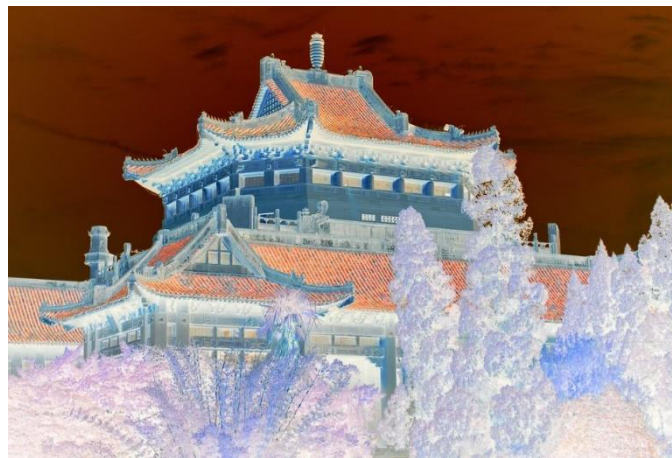
应用案例3：用NumPy处理图像



武汉大学
WUHAN UNIVERSITY

■ 负片效果

```
inverted_im = [255, 255, 255] - im_array  
Image.fromarray(inverted_im.astype('uint8')).show()
```





应用案例3：用NumPy处理图像



武汉大学
WUHAN UNIVERSITY

■ 图像亮度调整

```
# 调整亮度因子，范围可以是[0, 1]
brightness_factor = 0.2
# 每个像素值均加上brightness_factor * 255，增加亮度
brightened_im = im_array + brightness_factor * 255
brightened_im = np.clip(brightened_im, 0, 255).astype(np.uint8)
Image.fromarray(brightened_im).show()
```





应用案例3：用NumPy处理图像



操作图像的像素点

```
# 提取R通道
r_channel = im_array[:, :, 0]
# 调整G通道，将G通道的每个像素值乘以0.9，从而降低绿色的亮度
g_channel = (im_array[:, :, 1] * 0.9).astype(np.uint8)
# 处理B通道，对B通道进行二值化处理，所有像素值小于100的点被设为1，其余则为0
b_channel = (im_array[:, :, 2] < 100).astype(np.uint8)
# 合并通道
merged_im = np.stack([r_channel, g_channel, b_channel], axis=-1)
Image.fromarray(merged_im).show()
```

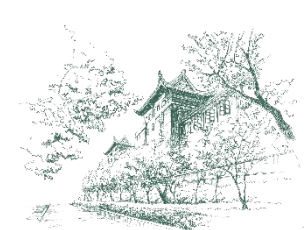




小结



- 数据结构在程序设计中扮演着组织和存储数据的关键角色，它是算法设计的基础，不同的数据结构会影响算法的实现方式、运行效率与存储空间需求。
- Python提供了丰富的数据结构来满足不同的编程需求。
- 列表是一种线性结构，可以当作数组，但比数组更实用（长度可变、元素类型可以不同），可以用来模拟队列和栈。
- 嵌套列表可以保存多维数据表。
- 可以通过列表推导式快速创建列表。
- 列表、元组、字符串都是序列，序列对象有一组公共的基本操作。
 - ✿ 列表是可变序列，元组和字符串是不可变序列。



小结



■ 怎样在列表与元组之间做出选择

✿ Python内部实现对元组做了大量优化，访问速度比列表更快。

➤ 如果定义了一系列常量值，主要用途仅是对它们进行遍历或其他“读”操作，而不需要对其元素进行任何修改，那么一般建议使用元组而不用列表。

✿ 元组在内部实现上不允许修改其元素值，从而使得代码更加安全。

➤ 例如，调用函数时使用元组传递参数可以防止在函数中修改元组对象，而使用列表则很难保证这一点。

■ 标准库的statistics模块提供了数据统计函数。

✿ 注意调用统计函数前按要求准备好数据。

■ 扩展库NumPy是Python用于科学计算的基础库。

✿ 多维数组是其主要的数据结构。



武汉大学

WUHAN UNIVERSITY

谢谢

THANK YOU

